

Experimentation protocol (working draft)

Unsupervised structural clustering of the eukaryotic virome

Pascal Nardi — M2 internship

June 16, 2026

Contents

1	General idea	2
2	Dataset	2
2.1	Source	2
2.2	Quality filtering	2
2.3	How each protein is represented	2
2.4	Train / validation split	3
3	Architecture	3
3.1	Features per rank	3
3.2	The attention layer	4
3.3	Schematic of the model	4
3.4	Hyperparameters	5
4	Training method	5
4.1	What contrastive learning is	5
4.2	SimCLR and the two views	6
4.3	The NT-Xent (InfoNCE) loss	6
4.4	Hard-negative batches	6
4.5	TM-score auxiliary term	6
5	Plan of experiments	7
5.1	Metrics	7
5.2	Baselines	7
5.3	Health checks I want to run every epoch	7
6	First results (preliminary)	8

1 General idea

Viral proteins change in sequence much faster than they change in shape. Because of that, the usual sequence-based homology searches miss real relationships that are still visible at the level of the 3D fold. The idea behind DeltaFold is to learn, without labels, an embedding space where two viral proteins end up close together when they share a fold, so that clustering the embeddings recovers known protein families and possibly suggests structural links that sequence methods cannot find.

So the biological question I would like the method to be able to answer is whether an unsupervised structural encoder can recover viral protein families, and whether it can connect families that have diverged in sequence but are still structurally related. The methodological question I would like your opinion on is whether the setup below is enough to tell apart real structural learning from the model just exploiting easy shortcuts or collapsing its representation.

2 Dataset

2.1 Source

I use the predicted structural proteome of the eukaryotic virome from Nomburg et al. [1]. The release has 67,715 predicted protein structures across 4,463 eukaryotic viral species. The structures are organised by family on disk, which gives me weak family labels. I only use those labels to evaluate the model, never during training.

2.2 Quality filtering

Each structure is a prediction, so confidence varies a lot. I read the per-residue pLDDT from the B-factor column of each structure and drop any protein whose mean pLDDT is below 70, which I found to be the usual cut for a confident prediction. I also leave out the proteins with no assigned family when building the labelled evaluation set. After filtering, around 67,000 proteins remain.

2.3 How each protein is represented

Before training, I convert every protein into a Protein Combinatorial Complex, the multi-rank topological structure introduced for protein representation learning by Topotein [2]. It organises a protein at four ranks:

- Rank 0, the residues, treated as nodes.
- Rank 1, the interactions, treated as edges.
- Rank 2, the secondary-structure elements (helices, strands, loops).
- Rank 3, the whole protein, as a single global node.

The rank-1 edges are built geometrically rather than from the sequence. For every residue I take the $C\alpha$ atom as its position, compute all pairwise $C\alpha$ - $C\alpha$ distances, and connect each residue to its 16 nearest neighbours in space (the residue itself is the closest point at distance zero, so it is excluded). This gives a directed k -nearest-neighbour graph with $k = 16$, so every residue has 16 outgoing edges to the residues that are physically closest to it, independent of where they sit along the chain. Each edge stores the $C\alpha$ - $C\alpha$ distance, a sinusoidal encoding of that distance, and the displacement vector between the two $C\alpha$ atoms.

The rank-2 elements are the secondary-structure segments, and they are built in three steps. First, every residue is assigned a secondary-structure state from the 3D coordinates using a DSSP-style annotation, reduced to three classes: helix, strand, and coil/loop. Second, consecutive residues that share the same state are grouped into one contiguous segment, so each helix, each strand and each loop becomes a single rank-2 element. Third, for every segment I take the $C\alpha$ coordinates of its residues and summarise its shape with a small principal-component analysis of their spatial spread: I compute the three eigenvalues of the coordinate covariance matrix (how extended the segment is along its three principal axes) and five shape descriptors derived from those eigenvalues (linearity, planarity, scattering, omnivariance and anisotropy). Together with a one-hot encoding of the segment type (helix / strand / loop), this gives the 12 features that describe each secondary-structure element. The mapping from each residue to the segment it belongs to is also stored, so the model knows which residues make up each element. The exact features carried by each rank are summarised in Section 3.

2.4 Train / validation split

The runs use a phylogenetic split: proteins are partitioned by taxonomic id so that the train and validation sets contain different viral taxa. The point is to stop the model from passing validation just by memorising taxon-level regularities.

One thing I want to raise: a phylogenetic split removes taxonomic leakage but not structural leakage. Two unrelated taxa can still share a fold, so some validation proteins will have a structural near-twin in the training set. I think this is acceptable because it reflects reality, but it means the validation numbers tell me “can the encoder place a known fold in the right place”, not “can it generalise to a fold it has never seen”. If we want the second claim, I would need to also hold out a set of folds entirely.

3 Architecture

The model is a higher-order attention network that operates directly on the Protein Combinatorial Complex, in the spirit of the higher-order attention networks of Hajij et al. [3] and the protein-specific topological network of Topotein [2]. It passes messages between the ranks of the complex (residues, edges, secondary structure, whole protein) and not only between residues.

3.1 Features per rank

The residue nodes are described by purely structural and geometric features: the 3Di structural-alphabet token from Foldseek [6] and the backbone dihedral angles. I deliberately leave out amino-acid identity and any sequence-order information, so the model has to rely on the geometry of the fold rather than on composition or position along the chain. The features that are actually used are:

Rank	Entity	Features used
0	Residue	3Di structural-alphabet token, backbone dihedrals (ϕ, ψ)
1	Edge	$C\alpha$ - $C\alpha$ distance, sinusoidal distance encoding, displacement vector
2	SSE	type (helix/strand/loop), 5 shape descriptors, 3 covariance eigenvalues (12 dims)
3	Protein	size, radius of gyration, 5 shape descriptors, 3 gyration eigenvalues (10 dims)

Each rank is mapped to a common width of 128 by a small two-layer perceptron before the attention layers.

3.2 The attention layer

The core operation of the model is *attention*, so it is worth saying plainly what that means before describing the layer. Attention is a way for one element to build an updated description of itself by taking a weighted average of information coming from a set of other elements, where the model itself decides how much weight each neighbour gets. Concretely, the element that is being updated emits a short vector called a *query*, every candidate neighbour emits a *key* and a *value*, and the weight given to a neighbour is large when its key matches the query and small otherwise. The updated element is then the weighted sum of the neighbours’ values. The useful property is that these weights are not fixed in advance: the model learns what makes a neighbour relevant, so a residue can learn to pay attention to the residues that matter for its local fold rather than to a fixed window of sequence neighbours.

The network stacks four identical layers. Within each layer, attention is applied four times, once for each direction of information flow between the ranks of the complex:

1. **Residues $\rightarrow$ edges.** Each edge looks at the residue it starts from and updates its own description. An edge therefore learns to summarise the local environment of the residue it connects.
2. **Residues $\rightarrow$ secondary-structure elements.** Each secondary-structure element looks at the residues that compose it and updates its description, so a helix or strand gathers information from its constituent residues.
3. **Edges $\rightarrow$ residues.** Each residue now looks back at the edges arriving at it and refines its own description from them. This is the step where a residue integrates information about its spatial neighbourhood.
4. **Residues and elements $\rightarrow$ whole protein.** The residue and secondary-structure descriptions are averaged and used to update a single global vector that represents the whole protein.

When an edge weighs its neighbours, I normalise the weights with a softmax (the standard way of turning match scores into weights that sum to one), and I add a small geometric bias that gives slightly more weight to spatially closer neighbours, so that physical proximity is built in and not only learned. After each of the four steps the updated description is combined with its previous value through a residual connection and a normalisation, which keeps training stable.

The final protein embedding is produced by averaging the residue descriptions, concatenating that with the global protein vector, passing the result through a last small perceptron, and normalising it to unit length so that every protein is a point on a unit sphere. Distances on that sphere are what I later cluster.

3.3 Schematic of the model

Figure 1 sketches one attention layer in the tensor-diagram style used in the HOAN paper [3]: each box is a tensor (the feature array of one rank, with its dangling wire standing for the feature dimension), and each arrow is a learned attention map that sends information from one rank to another. The four arrows are the four steps above; the whole block is repeated four times, and a final readout turns the residue and protein tensors into the output embedding.

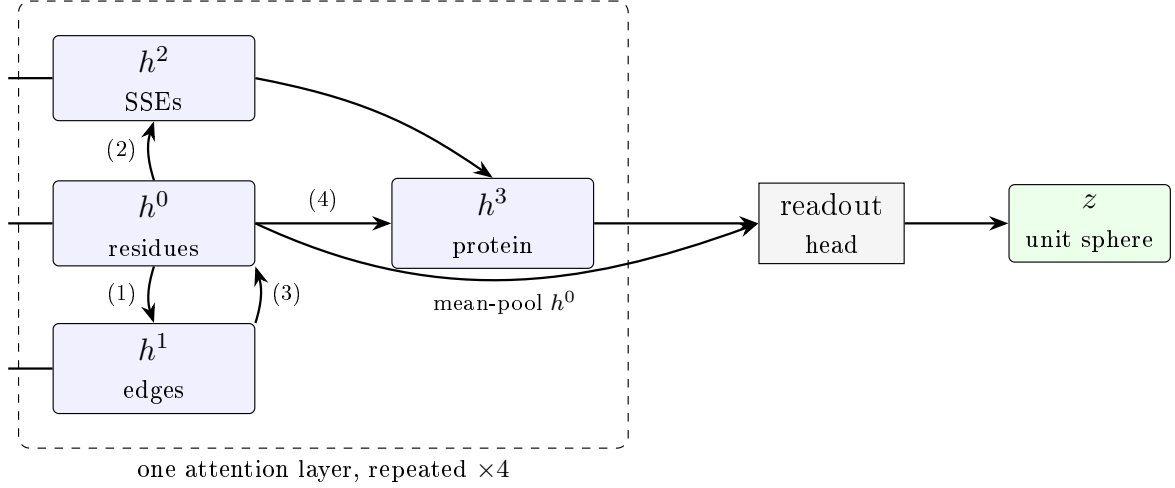


Figure 1: One attention layer of the model in tensor-diagram style. Boxes are the per-rank feature tensors $h^0 - h^3$ (residues, edges, SSEs, whole protein); arrows are learned attention maps. The numbered arrows are the four steps of Section 3.2: (1) residues $\rightarrow$ edges, (2) residues $\rightarrow$ SSEs, (3) edges $\rightarrow$ residues, (4) pooling to the global protein vector. The block is applied four times, then a readout head combines the pooled residue tensor and the protein tensor into the unit-length embedding z .

3.4 Hyperparameters

Group	Hyperparameter	Value
Model	embedding width	128
	attention layers	4
	dropout	0.1
	graph degree k	16
	output dim	128, ℓ_2 -normalised
Optim	optimizer	AdamW
	learning rate	1×10^{-4}
	weight decay	1×10^{-5}
	scheduler	reduce-on-plateau (factor 0.5, patience 2)
	gradient clip	value clip 1.0
	epochs	50
	batch size	64 proteins
	residue budget / batch	3500
Loss	contrastive temperature τ	0.1
	TM-score auxiliary weight	0.1

4 Training method

4.1 What contrastive learning is

The model is trained without any family labels, using contrastive learning. The principle is simple: I take each protein and make two slightly perturbed copies of it, then I train the encoder so that the two copies of the same protein land close together in the embedding space while copies of different proteins are pushed apart. Over many proteins, the only way to satisfy this everywhere is to place proteins that are genuinely alike near each other and proteins that differ far apart. The

encoder therefore discovers, on its own, the features that make two proteins similar, without ever being told what the families are.

4.2 SimCLR and the two views

The exact recipe I follow is SimCLR [4], a standard contrastive framework. For each protein I generate two augmented “views” of its complex. The augmentations are meant to act like measurement noise rather than to change the fold: I add small Gaussian noise (about 0.3 to 0.5 Å) to the C α coordinates and recompute the local graph distances, and I randomly mask a fraction of the residue features. The two views of one protein form a positive pair that should be pulled together; every other protein in the batch provides negative examples that should be pushed away.

4.3 The NT-Xent (InfoNCE) loss

The objective that turns this idea into a number to minimise is the NT-Xent loss, the normalised temperature-scaled cross-entropy used in SimCLR, which is an instance of the InfoNCE objective [5, 4]. For a batch of N proteins there are $2N$ views. Writing z_i for the unit-length embedding of view i , and $\text{sim}(u, v) = u^\top v$ for the cosine similarity of two unit-length vectors, the loss for a positive pair (i, j) (the two views of the same protein) is

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)},$$

and the total loss is the average of $\ell_{i,j}$ over all positive pairs in the batch. The numerator rewards the model for making the two views of the same protein similar; the denominator penalises it for making a view similar to any other protein in the batch. The temperature τ (set to 0.1) controls how sharply the model focuses on the hardest negatives: a smaller τ makes the loss pay more attention to negatives that are already close by.

4.4 Hard-negative batches

A contrastive model only learns something useful if the negatives are hard. If the other proteins in a batch are obviously different, the model can tell them apart using a cheap cue (for instance, just the protein length) and never has to look at the fold. To prevent that, I build the batches deliberately: proteins are binned by length and then ordered within a bin by their secondary-structure composition, so each batch groups proteins that look superficially similar but have a different topology. This forces the model to use the actual fold to separate them. Batches are budgeted by total residue count rather than by a fixed number of proteins, which keeps the per-step cost bounded on the 16 GB machine.

4.5 TM-score auxiliary term

On top of the contrastive loss I add a small auxiliary term, weighted 0.1, that regresses the true pairwise TM-score of sampled protein pairs (precomputed and cached). TM-score is a standard measure of structural similarity between two proteins, so this term ties the geometry of the embedding directly to a known structural quantity, rather than letting it be shaped by the contrastive objective alone.

5 Plan of experiments

5.1 Metrics

I evaluate the embeddings against the held-out family labels and against ground-truth structural similarity. I report the adjusted Rand index of k -means clusters, the adjusted Rand index of HDBSCAN (density-based) clusters [7], the normalised mutual information with the family labels, the TM-recall at k (the fraction of a protein’s true structural homologs that land among its k nearest neighbours in the embedding), and TM- ρ (the correlation between embedding distance and pairwise TM-score). I think TM-recall is the cleanest of these because it does not depend on a clustering algorithm.

The adjusted Rand index (ARI) compares the clustering of the embeddings to the known families, corrected for what you would get by chance, on a scale where 0 is random and 1 is perfect. It wobbles for two reasons. First, it depends on a clustering step that is itself unstable: k -means starts from a random initialisation, and HDBSCAN is density-based, so a small change in the embedding can merge or split clusters and move the score by several points. Second, the problem is hard and very imbalanced (around 2700 clusters with about 40% singletons), so the score reacts to a handful of borderline proteins changing cluster. This is why the HDBSCAN ARI swings more than the k -means one: it is the more sensitive of the two to small changes in the geometry.

The normalised mutual information (NMI) also compares the clustering to the families, but instead of matching cluster assignments it asks how much knowing a protein’s cluster tells you about its family, again on a 0 to 1 scale. It is the smoothest of the three, staying around 0.86 to 0.885, because it is carried by the many easy, well-separated clusters and is less sensitive than ARI to where exactly the borderline proteins fall. Its slow upward drift is the clearest hint of gradual improvement, but the steps are small and sit within the run-to-run noise, so I would not over-read it on its own.

TM- ρ is different in kind: it is a rank correlation between distance in the embedding and the TM-score (a standard measure of structural similarity) over a sample of protein pairs. Because it does not involve any clustering, it is the most stable of the three, sitting between about -0.75 and -0.80 across every epoch. A negative value is what we want, since structurally similar proteins (high TM-score) should end up close together (small distance). The small wobble comes mostly from a different set of pairs being sampled each epoch rather than from real changes in the model, and the fact that it barely moves suggests the structural signal was picked up early and then stayed roughly constant.

The practical consequence is that no single epoch is a reliable readout. Reading off epoch 20 alone would overstate things, because epoch 20 happens to be a local high for the HDBSCAN ARI. A fairer summary is the level and trend over the last several epochs, and ideally an average over a few random seeds. This is the main reason I want the per-epoch health checks of Section 5.3 and repeated runs before drawing any conclusion from these numbers.

5.2 Baselines

I want every number reported against two references on the same split: a random or shuffled-embedding baseline as a floor, and Foldseek [6] as the method to beat. A result only means something relative to both.

5.3 Health checks I want to run every epoch

Because a contrastive encoder can cheat by collapsing all embeddings toward a common direction, I want to log a few diagnostics every epoch and treat them as gating criteria rather than something

I look at afterwards. These are the spread of the embedding values per dimension (it should stay clearly above the collapse threshold of about 0.02), the mean off-diagonal cosine similarity (it should stay low; values near 0.9 to 0.97 mean the vectors are nearly parallel), and the effective rank of the embedding covariance as a single number for how many dimensions are actually being used. I would only interpret a checkpoint biologically if these are healthy at that epoch.

6 First results (preliminary)

These numbers come from a single run on the 16 GB machine. The training loss converges very quickly: it falls from about 2.5 at the start of training to about 0.5 by the end of the first epoch, and after that it barely moves. The validation loss behaves the same way, hovering around 0.40 for the rest of the run. In other words the contrastive objective is essentially solved within the first epoch.

What is more interesting is that the evaluation metrics keep improving a little even after the loss has flattened. Pushing the run from epoch 6 to epoch 20 still buys a small but consistent gain in clustering quality: the HDBSCAN ARI rises from about 0.25 to about 0.40 and the NMI from about 0.87 to 0.89, while TM- ρ decreases (becomes more negative), which means a stronger and therefore better anti-correlation between embedding distance and structural similarity. So the embedding geometry is still being refined after the loss plateaus, even though the gains are modest and noisy from epoch to epoch. Table 1 lists the values at the start of training, at epoch 6, and at epoch 20.

Metric	Initial	Epoch 6	Epoch 20
Training loss	≈ 2.5	0.53	0.43
Validation loss	—	0.41	0.38
ARI (HDBSCAN)	0.16	0.25	0.40
NMI (HDBSCAN)	0.86	0.87	0.89
TM- ρ	-0.75	-0.79	-0.78

Table 1: Preliminary results from a single run. “Initial” is the start of training for the losses and the first full-scale evaluation (epoch 5) for the clustering metrics; the earlier epochs were scored on a small development subset that is not comparable and are omitted, and the validation loss is only recorded from epoch 6. The training loss is essentially converged by epoch 6, yet the clustering metrics continue to improve slightly through epoch 20.

References

- [1] J. Nomburg, E. E. Doherty, N. Price, D. Bellieny-Rabelo, Y. K. Zhu, J. A. Doudna. Birth of protein folds and functions in the virome. *Nature*, 633:710–717, 2024. <https://doi.org/10.1038/s41586-024-07809-y>.
- [2] Z. Wang, A. R. Jamasb, et al. Topotein: Topological Deep Learning for Protein Representation Learning. *arXiv preprint arXiv:2509.03885*, 2025.
- [3] M. Hajij, G. Zamzmi, T. Papamarkou, N. Miolane, A. Guzmán-Sáenz, K. N. Ramamurthy. Higher-Order Attention Networks. *arXiv preprint arXiv:2206.00606*, 2022.
- [4] T. Chen, S. Kornblith, M. Norouzi, G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). *Proceedings of ICML*, 2020. arXiv:2002.05709.

- [5] A. van den Oord, Y. Li, O. Vinyals. Representation Learning with Contrastive Predictive Coding (InfoNCE). *arXiv preprint* arXiv:1807.03748, 2018.
- [6] M. van Kempen, S. S. Kim, C. Tumescheit, M. Mirdita, J. Lee, C. L. M. Gilchrist, J. Söding, M. Steinegger. Fast and accurate protein structure search with Foldseek. *Nature Biotechnology*, 42:243–246, 2024.
- [7] R. J. G. B. Campello, D. Moulavi, J. Sander. Density-Based Clustering Based on Hierarchical Density Estimates (HDBSCAN). *Advances in Knowledge Discovery and Data Mining (PAKDD)*, 2013.